CSM GLEAMING Solutions

COMPUTER SCIENCE MENTORS 61A

November 5, 2026 – October 9, 2026

1 Gleam basics

Gleam is a functional programming language. In computer science, functional programming is a paradigm where programs are constructed by applying and composing functions, and func- tion definitions are trees of expressions that map values to other values, rather than a sequence of statements which update the running state of the program. Gleam is a statically-typed, type-safe language. Type safety is the extent to which a programming language discourages or prevents type errors. The behaviors classified as type errors by a given programming language are usually those that result from attempts to perform operations on values that are not of the appropriate data type, e.g. trying to add a string to an integer. Type enforcement can be static (catching potential errors at compile time), dynamic (associating type information with values at run-time and consulting them as needed to detect imminent errors), or a combination of both. In the context of static type systems, type safety usually involves a guarantee that the eventual value of any expression will be a legitimate member of that expression's static type. When writing functions that operate on a given input, Gleam requires that you handle any theoretically possible output case, even if you've potentially handled such a case earlier. For instance, the function list.max (which behaves very similarly to Python's max function) does not return an integer directly, but a Result. This handles the case in which an empty list has been passed into max: it returns Ok(value) or Error(Nil) regardless of whether you've already handled the empty case earlier in your code, and are thus guaranteeing that it will always be an Ok result. When writing a case statement in Gleam, you must handle every theoretically possible case, even if that case may not be possible in the context of your own code. For instance, say you have a function that exclusively outputs even integers, and you use one of those integers n in a case statement:

```
case n \% 2 {
    0 -> //some code for the case in which n is even
    1 -> //some code for the case in which n is odd
    }
```

Gleam will require you to include the latter case, in which n your integer n could possibly be odd.

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
pub fn multiply(n: Int, m: Int) -> Int {
  case n > 0 {
    True -> _______ + ___________________________________
    False -> _______
  }
}
```

```
pub fn multiply(n: Int, m: Int) -> Int {
  case n > 0 {
    True -> m + multiply(n - 1, m)
    False -> 0
  }
}
```

2. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> _______ * ___________________________________
    False -> _______
  }
}
```

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> n * factorial(n - 1)
    False -> 1
  }
}
```

3. Implement `sum`, a tail recursive function that returns the sum of all elements of a list of integers.

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> _______
    [first] -> _______ + _______
    [first, ..rest] -> ___________________________________
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(_______, _______)
}
```

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> curr
    [first] -> curr + first
    [first, ..rest] -> sum_helper(rest, curr + first)
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(l, 0)
}
```

4. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> _______
    [first] -> ___________________________________
    [first, ..rest] -> ___________________________________
  }
}
```

```
fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> start
    [first] -> f(start, first)
    [first, ..rest] -> reduce(rest, f, f(start, first))
  }
}
```

5. Implement `prune_even`, which prunes every branch in a tree that has an even label. Odd child branches of branches with even labels are not preserved.

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even = list.filter(branches, _____________________________)
    let prune_children = list._________(_____________________________)

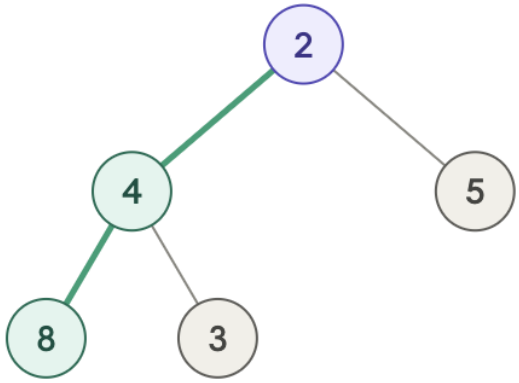
    Tree(label, prune_children)
}
```

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even =
        list.filter(branches, fn(branch) { branch.label % 2 != 0 })
    let prune_children = list.map(remove_even, prune_even)

    Tree(label, prune_children)
}
```

6. A *treemil* is a tree in which at least one of the child node(s) of each non-leaf node has a label that is a multiple of the label of the node. Alternatively, a treemil is a tree where every node has at least one path from the node to a leaf consisting entirely of nodes that are multiples of its parent. Implement `is_treemil`, which returns whether a tree is a treemil.




```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    // valid if:
    // 1. at least one child node is a multiple of its parent
    // 2. all children are valid treemils
    // leaves are valid treemils

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(label, branches) -> {
            let child_multiples = list._______(_______, ________________)
            case list._________(child_multiples) >= 1 {
                True -> list._________(___________________________________)
                False -> _________
            }
        }
    }
}
```

```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    case tree {
        Tree(_, []) -> True

        Tree(label, branches) -> {
            let child_multiples =
                list.filter(branches, fn(b) { b.label % label == 0 })
            case list.length(child_multiples) >= 1 {
                True -> list.all(list.map(branches, is_treemil), fn(x) { x
                    })
                False -> False
            }
        }
    }
}
```

7. Implement `max_treemil` which takes in a tree `t` and determines the highest number of nodes in a treemil that is a subtree of `t`. You may assume `is_treemil` is implemented correctly from the previous question.

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    //Given a tree (that is not necessarily a treemil), returns
    //the highest number of nodes in a treemil in the tree.

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(_, branches) -> {
            case ___________________________________ {
                True -> 1 + int._________(___________________________________)
                False -> result.unwrap(list.max(_________, int.compare), 0)
            }
        }
    }
}
```

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    case tree {
        Tree(_, []) -> 1

        Tree(_, branches) -> {
            case is_treemil(tree) {
                True -> 1 + int.sum(list.map(branches, max_treemil))
                False -> {
                    let sizes = list.map(branches, max_treemil)
                    result.unwrap(list.max(sizes, int.compare), 0)
                }
            }
        }
    }
}
```